

GymIO

Sistema de Gestão Interna para Academias

Documentação da conexão com o Supabase
e do processo de autenticação no login

2026

1. Objetivo da Integração

A integração com o Supabase foi realizada para centralizar o banco de dados PostgreSQL e utilizar o serviço de autenticação da plataforma. No GymIO, o front-end React não acessa diretamente o Supabase para realizar o login. A comunicação passa pelo back-end em Node.js/Express, que recebe as credenciais, solicita a autenticação ao Supabase e controla a sessão da aplicação.

2. Arquitetura da Comunicação

```
React (Vite) → Axios → API Express → Supabase Auth / PostgreSQL
```

O front-end utiliza Axios configurado com **withCredentials: true**, permitindo o envio dos cookies da sessão nas requisições. A API está configurada para receber requisições da aplicação React em **http://localhost:5173** e aceitar credenciais.

3. Configuração da conexão com o Supabase

No back-end, o pacote **@supabase/supabase-js** é utilizado para criar o cliente de comunicação com o projeto Supabase. A URL do projeto e a chave de acesso são obtidas por variáveis de ambiente.

O arquivo **backend/config/supabase.js** concentra essa configuração e exporta o cliente utilizado pelos controllers e pelo middleware de autenticação.

Variáveis utilizadas

Variável	Função
SUPABASE_URL	Identifica a URL do projeto Supabase.
SUPABASE_SERVICE_ROLE_KEY	Chave utilizada pelo back-end para acessar os serviços do Supabase.

4. Estrutura dos usuários

A autenticação e os dados administrativos são mantidos em camadas relacionadas. O Supabase Auth identifica o usuário por um **UUID**. Esse identificador é armazenado na tabela **pessoa** por meio do campo **auth_id**. A partir dele, o sistema consegue relacionar a identidade autenticada aos dados de pessoa e funcionário cadastrados no banco.

5. Processo de Login

O login foi implementado seguindo as etapas abaixo:

1. O usuário informa e-mail e senha na tela de login do React.
2. O front-end chama o serviço `authService.login()`, que envia os dados para `POST /api/auth/login`.
3. O back-end recebe as credenciais no `auth.controller.js`.
4. O controller chama `supabase.auth.signInWithPassword({ email, password })`.
5. O Supabase valida as credenciais. Se forem inválidas, a API retorna HTTP 401.
6. Se forem válidas, o back-end recebe a sessão e o `access_token`.
7. O `access_token` é armazenado em um cookie chamado `token`, configurado como `httpOnly`.
8. O front-end chama `GET /api/auth/me` para confirmar a sessão e carregar os dados do usuário.
9. O middleware `auth.js` lê o cookie, valida o token com `supabase.auth.getUser(token)` e disponibiliza o usuário em `req.user`.
10. A rota `/auth/me` consulta a tabela `persona` usando `auth_id` e retorna os dados necessários ao front-end.

6. Controle das rotas protegidas

As rotas que exigem autenticação podem utilizar o middleware **auth.js**. O middleware verifica primeiro se existe o cookie **token**. Caso não exista, a requisição recebe HTTP 401. Quando existe, o token é enviado ao Supabase por meio de **getUser()**. Se a validação falhar, a requisição também é recusada.

7. Manutenção da sessão no React

O **AuthContext** centraliza o estado de autenticação do front-end. Ao iniciar a aplicação, o contexto executa **GET /auth/me**. Se a sessão estiver válida, o usuário retornado pela API é armazenado no estado. O componente **ProtectedRoute** utiliza esse estado para impedir o acesso às páginas protegidas quando não há usuário autenticado.

8. Logout

No logout, o front-end chama **POST /api/auth/logout**. O back-end remove o cookie **token** com `clearCookie()` e o **AuthContext** limpa o usuário armazenado no estado da aplicação.

9. Fluxo resumido

LOGIN: React → POST /api/auth/login → Supabase Auth → access_token → cookie token → GET /api/auth/me → usuário autenticado

REQUISIÇÃO PROTEGIDA: React → cookie token → middleware auth.js → Supabase getUser(token) → req.user → controller

10. Tecnologias envolvidas

Tecnologia	Utilização
React + Vite	Interface e gerenciamento do estado de autenticação.
Axios	Comunicação do front-end com a API e envio de cookies.
Node.js + Express	Back-end responsável pelas rotas e regras de autenticação.
Supabase Auth	Validação das credenciais e identificação do usuário.

PostgreSQL / Supabase	Armazenamento dos dados relacionados a pessoa e funcionário.
Cookie HTTP	Transporte do access_token entre navegador e API.

Observação técnica: esta documentação descreve a implementação encontrada no projeto GymIO. Em produção, a configuração do cookie deve utilizar HTTPS e **secure: true**. Também é recomendável não armazenar senhas em texto puro na tabela de aplicação; a autenticação deve permanecer sob responsabilidade do Supabase Auth.